

Deep Generative Models

Chapter 5: Variational Inference and Variational Autoencoding

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Summer 2026

Bayesian Setting: Evidence

Let's rewrite the **challenge** in a Bayesian setting: we know the **prior** latent distribution $P(z)$ and **generator** $P(x|z)$ looking for the so-called **evidence**

Evidence

In Bayesian formulation, the distribution of a known sample x **marginalized** over the **latent** is called **evidence**

$$P(x) = \int P(x|z)P(z) dz$$

- + Isn't this **evidence** simply the **likelihood**?!
- Sure! When it is computed with a **learnable** model $P_{\mathbf{w}}(x|z)$, it computes the **likelihood** of the **model**

Computing Evidence: *Direct Approach*

Complexity of *direct evidence computation* is *exponential* in *latent size*

Illustration. Consider a *discrete* latent $z \in \mathbb{Z}^m$ with $\mathbb{Z}$ having C elements: the *evidence* in this case is computed as

$$P(x) = \sum_z P(x|z) P(z)$$

which requires C^m additions!

Moral of Story

Direct computation of the *evidence* is *not tractable* even though we know both *prior* $P(z)$ and *generator* $P(x|z) \equiv$ *direct* sampling from a *general distribution*

Computing Evidence: Monte Carlo

An alternative approach is to **estimate evidence** via **Monte Carlo**: we note that

$$P(x) = \int P(x|z)P(z) dz = \mathbb{E}_{z \sim P(z)} \{P(x|z)\}$$

We can hence **estimate** the **evidence** as

$$\hat{P}(x) = \hat{\mathbb{E}}_{z \sim P(z)} \{P(x|z)\} = \frac{1}{n} \sum_i P(x|z_i)$$

for samples $z_i \sim P(z) \rightsquigarrow$ this is **tractable** as we have $P(z)$ and $P(x|z)$, but

↳ For large enough $P(x|z)$ we need **samples** z_i for which x is **likely**

↳ **Only few** samples $z_i \sim P(z)$ would have **useful** $P(x|z_i)$

Moral of Story

MC estimate is **high variance** and needs a **huge** set of samples, i.e., huge n

Computing of Evidence by Importance Sampling

There is an **intuitive** remedy to **reduce** variance of **MC estimate**: assume we know $Q(z|x)$ whose **samples are good** for the given x ; then,

we may use **samples** $z_i \sim Q(z|x)$ to estimate **evidence**

- + But in **marginalization**, we average over $P(z)$ not some $Q(z|x)$! Right?!
- Right! We can do a **simple** trick!

$$\begin{aligned} P(x) &= \int P(x|z)P(z) \, dz = \int P(x|z) \frac{P(z)}{Q(z|x)} Q(z|x) \, dz \\ &= \mathbb{E}_{z \sim Q(z|x)} \left\{ P(x|z) \frac{P(z)}{Q(z|x)} \right\} \end{aligned}$$

We can now use **samples of** $Q(z|x)$ to estimate **evidence** $P(x)$

Importance Sampling

Importance Sampling

We can estimate the *evidence* as

$$P(x) = \hat{\mathbb{E}}_{z \sim Q(z|x)} \left\{ P(x|z) \frac{P(z)}{Q(z|x)} \right\} = \frac{1}{n} \sum_i P(x|z_i) \frac{P(z_i)}{Q(z_i|x)}$$

with z_i sampled from our *good distribution* $z_i \sim Q(z|x)$

The core idea is that *samples* $z_i \sim Q(z|x)$ are *more important*

↳ $z_i \sim Q(z|x)$ are in the region where x is *likely*

↳ Samples $z_i \sim Q(z|x)$ result in *useful* samples for estimating $P(x|z_i)$

Thus, the *evidence* estimator has *less variance*

Importance Sampling: Good Distribution for Sampling

- + But, what is this *good distribution* $Q(z|x)$?!
- Ideally, the *posterior* $P(z|x)$!

If we set $Q(z|x) = P(z|x)$; then, each sample reads

$$P(x|z_i) \frac{P(z_i)}{P(z_i|x)} = \frac{P(x, z_i)}{P(z_i|x)} = P(x)$$

So, only *single sample* is enough to estimate *evidence* with *zero variance*!

Good Distribution for Importance Sampling

A good choice of $Q(z|x)$ is the one that is *close to posterior* $P(z|x)$

Importance Sampling: *Learning Objective*

! Attention

Note that the *posterior* is given as

$$P(z|x) = \frac{P(x|z)P(z)}{\boxed{P(x)} \rightarrow \text{evidence}}$$

whose computation is *as complex as computing evidence*

- + How should we find a *good* $Q(z|x)$ at the end?!
- We try to learn it by *approximating* $P(z|x)$!
- + But how can we do this?! We don't know $P(z|x)$ in the first place!
- We do it by an *implicit approach* $\rightsquigarrow$ *variational inference*

Variational Inference: *Right Distribution*

Let's say we have a **class of distributions** Q defined on **latent space**¹: we can **learn** the **best choice of Q** for importance sampling as

$$Q_{|x}^* = \operatorname{argmin}_{Q_{|x}} D_{\text{KL}}(Q_{|x} \| P_{|x})$$

$Q_{|x} \rightarrow Q(\cdot|x)$

The **key challenge** is though that we **don't know** $P(\cdot|x)$

Let's expand this divergence a bit

$$\begin{aligned} D_{\text{KL}}(Q_{|x} \| P_{|x}) &= \mathbb{E}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x)}{P(z|x)} \right\} = \mathbb{E}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x)P(x)}{P(z|x)P(x)} \right\} \\ &= \mathbb{E}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x)P(x)}{P(x, z)} \right\} \end{aligned}$$

¹We later **learn** Q by a **computational model**

Variational Inference: *Right Distribution*

We can now use the *chain rule* to write

$$\begin{aligned} D_{\text{KL}}(Q_{|x} \| P_{|x}) &= \mathbb{E}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x) P(x)}{P(x,z)} \right\} \\ &= \mathbb{E}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x) P(x)}{P(x|z) P(z)} \right\} \\ &= \mathbb{E}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x)}{P(z)} + \log \frac{1}{P(x|z)} + \log P(x) \right\} \end{aligned}$$

Let's look at each expression inside the expectation *individually*

Variational Inference: Right Distribution

The first term is a *KL divergence*

$$\mathbb{E}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x)}{P(z)} \right\} = D_{\text{KL}} (Q_{|x} \| P_z)$$

which we can *estimate* easily

- 1 Collect samples $z_i \sim Q(z|x)$
- 2 Compute the distribution at each *sample*, i.e., $Q(z_i|x)$
- 3 Compute *latent distribution* at each *sample* as well, i.e., $P(z_i)$

The *estimate* of *this term* is then given by

$$\hat{D}_{\text{KL}} (Q_{|x} \| P_z) = \hat{\mathbb{E}}_{z \sim Q_{|x}} \left\{ \log \frac{Q(z|x)}{P(z)} \right\} = \frac{1}{n} \sum_i \log \frac{Q(z_i|x)}{P(z_i)}$$

Variational Inference: *Right Distribution*

The second term is

$$\mathbb{E}_{z \sim Q|x} \left\{ \log \frac{1}{P(x|z)} \right\} = -\mathbb{E}_{z \sim Q|x} \{ \log P(x|z) \}$$

which we can again *easily estimate*

- 1 Collect samples as $z_i \sim Q(z|x)$
- 2 Compute generator at each *sample*, i.e., $P(x|z_i)$

We then *estimate this term* as

$$-\hat{\mathbb{E}}_{z \sim Q|x} \{ \log P(x|z) \} = -\frac{1}{n} \sum_i \log P(x|z_i)$$

Variational Inference: *Right Distribution*

The third term is indeed the *log-evidence*

$$\begin{aligned}\mathbb{E}_{z \sim Q_{|x}} \{\log P(x)\} &= \int \log P(x) Q(z|x) \, dz \\ &= \log P(x) \underbrace{\int Q(z|x) \, dz}_1 = \log P(x)\end{aligned}$$

Putting all three terms together

$$D_{\text{KL}}(Q_{|x} \| P_{|x}) = D_{\text{KL}}(Q_{|x} \| P_z) - \mathbb{E}_{z \sim Q_{|x}} \{\log P(x|z)\} + \log P(x)$$

This offers a *tractable* way for estimating the *right distribution*!

ELBO: Evidence Lower Bound

Recall that we were looking for

$$Q_{|x}^* = \underset{Q_{|x}}{\operatorname{argmin}} D_{\text{KL}} (Q_{|x} \| P_{|x})$$

We can use the expansion to write

$$\begin{aligned} Q_{|x}^* &= \underset{Q_{|x}}{\operatorname{argmin}} D_{\text{KL}} (Q_{|x} \| P_z) - \mathbb{E}_{z \sim Q_{|x}} \{\log P(x|z)\} + \log P(x) \\ &= \underset{Q_{|x}}{\operatorname{argmin}} D_{\text{KL}} (Q_{|x} \| P_z) - \mathbb{E}_{z \sim Q_{|x}} \{\log P(x|z)\} \\ &= \underset{Q_{|x}}{\operatorname{argmax}} \boxed{\mathbb{E}_{z \sim Q_{|x}} \{\log P(x|z)\} - D_{\text{KL}} (Q_{|x} \| P_z)} \end{aligned}$$

The *objective* on this *maximization* can be *estimated* $\rightsquigarrow Q_{|x}$ can be learned!

ELBO: Evidence Lower Bound

ELBO: Evidence Lower Bound

For a given $Q_{|x}$, the ELBO is defined as

$$\text{ELBO} (Q_{|x}) = \mathbb{E}_{z \sim Q_{|x}} \{ \log P(x|z) \} - D_{\text{KL}} (Q_{|x} \| P_z)$$

which can be *estimated* by *sampling* from $Q_{|x}$ as

$$\hat{\text{ELBO}} (Q_{|x}) = \hat{\mathbb{E}}_{z \sim Q_{|x}} \{ \log P(x|z) \} - \hat{D}_{\text{KL}} (Q_{|x} \| P_z)$$

The *key feature* of *ELBO* is that it is *maximized* via the *distribution* that lies in *minimum KL divergence* of *posterior* $P(z|x)$, i.e.,

$$\underset{Q_{|x}}{\text{argmin}} D_{\text{KL}} (Q_{|x} \| P_{|x}) = \underset{Q_{|x}}{\text{argmax}} \text{ELBO} (Q_{|x})$$

ELBO Properties: *Bounding Evidence*

It is easy to see that **ELBO** is really a **lower bound on log-evidence**: recall

$$D_{\text{KL}}(Q_{|x} \| P_{|x}) = \underbrace{D_{\text{KL}}(Q_{|x} \| P_z) - \mathbb{E}_{z \sim Q_{|x}} \{\log P(x|z)\}}_{-\text{ELBO}(Q_{|x})} + \log P(x)$$

We can sort things out and write

$$\log P(x) = D_{\text{KL}}(Q_{|x} \| P_{|x}) + \text{ELBO}(Q_{|x})$$

No matter what $Q_{|x}$ is $\rightsquigarrow$ we always have $D_{\text{KL}}(Q_{|x} \| P_{|x}) \geq 0$

ELBO: *Bounding Evidence*

For **any** distribution $Q_{|x}$, **ELBO** bounds the **log-evidence** from below, i.e.,

$$\log P(x) \geq \text{ELBO}(Q_{|x})$$

ELBO Properties: *Optimal ELBO* $\equiv$ *Log-Likelihood*

Let's look again at the identity *and think about ideal case*

$$\log P(x) = D_{\text{KL}}(Q_{|x} \| P_{|x}) + \text{ELBO}(Q_{|x})$$

if *ELBO* is *ideally* optimized $\rightsquigarrow D_{\text{KL}}(Q_{|x}^* \| P_{|x}) = 0$

↳ This means that the *optimal ELBO* touches *log-evidence*

↳ Recall that *log-evidence* computes *log-likelihood* for us

ELBO: *Optimal ELBO* $\equiv$ *Log-Likelihood*

Assuming *true posterior* $P_{|x}$ belongs to class of distributions described by $Q_{|x}$

$$\log P(x) = \max_{Q_{|x}} \text{ELBO}(Q_{|x})$$

Variational Inference: Wrap Up

Let us now summarize: consider a setting in which we know

- 1 *prior latent* distribution $P(z)$, and
- 2 the conditional *generator* $P(x|z)$

We want to compute the *evidence* $\equiv$ *likelihood* $P(x)$

Solution via Variational Inference

- 1 Find a *good estimator* of *posterior* by maximizing the *ELBO*

$$Q_{|x}^* = \operatorname{argmax}_{Q_{|x}} \text{ELBO}(Q_{|x})$$

- 2 Use importance sampling to compute the *evidence*

$$P(x) = \mathbb{E}_{z \sim Q_{|x}^*} \left\{ P(x|z) \frac{P(z)}{Q^*(z|x)} \right\}$$

Approximating Evidence Computationally

- + How can we implement *variational inference* in *practice*?
- We use a *computational model*!

We first consider a *computational model* for $Q_{|x} \equiv Q_{\mathbf{w}}$

$\text{VI}(P_z:\text{latent prior}, P_{x|z}:\text{model}, x:\text{data})$

- 1: Let $Q_{\mathbf{w}}$ be a *computational model* for latent distribution
- 2: **for** multiple epochs **do**
- 3: Sample a batch of *latent samples* $\{z^j : j = 1, \dots, n\}$ from $Q_{\mathbf{w}}$
- 4: **for** $j = 1, \dots, n$ **do**
- 5: Compute $\text{ELBO}^j = \log P(x|z^j) - \log \frac{Q_{\mathbf{w}}(z^j)}{P(z^j)}$
- 6: Backpropagate over $Q_{\mathbf{w}}$ to compute $\nabla_{\mathbf{w}} \text{ELBO}^j$
- 7: **end for**
- 8: Update $\mathbf{w}$ using $\text{Opt_avg} \{-\nabla_{\mathbf{w}} \text{ELBO}^j\}$
- 9: **end for**
- 10: **return** *trained distribution* $Q_{\mathbf{w}}$

Approximating Evidence Computationally

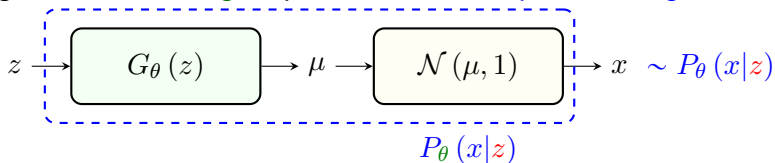
We then use the *trained model* to estimate *evidence* by *importance sampling*

VI_Evidence(Q_w)

- 1: Sample a batch of *latent samples* $\{z^j : j = 1, \dots, n\}$ from Q_w
- 2: **for** $j = 1, \dots, n$ **do**
- 3: Compute $\hat{P}^j = P(x|z^j) \frac{P(z^j)}{Q_w(z^j)}$
- 4: **end for**
- 5: Estimate evidence as $\hat{P}(x) \leftarrow \text{mean} \{ \hat{P}^j \}$
- 6: **return** *Evidence estimator* $\hat{P}(x)$

Back to MLE: Probabilistic Generator

Let's get back to our **original** problem: we have a **probabilistic generator**



Mean μ is computed from latent $z \rightsquigarrow x$ is sampled from $\mathcal{N}(\mu, 1)$

Latent-space Generator

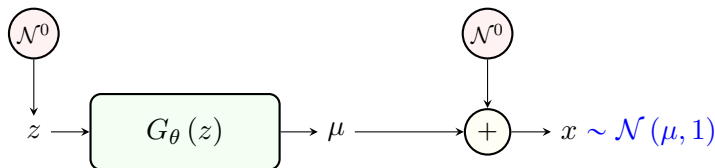
The sample x is generated conditionally from **latent** z as

$$P_\theta(x|z) = \mathcal{N}(G_\theta(z), 1)$$

Building Probabilistic Generator

It is easy to build such a model via a neural network

- 1 Sample **latent** as $z \sim \mathcal{N}(0, 1) \equiv \mathcal{N}^0$
 - ↳ Simply generate m independent Gaussian scalars
- 2 Pass forward **latent** through **the model** as $\mu = G_\theta(z)$
- 3 Sample **data** as $x \sim \mathcal{N}(\mu, 1)$
 - ↳ Simply sample $\varepsilon \sim \mathcal{N}^0$
 - ↳ Compute $x = \mu + \varepsilon$



MLE Training: *Formulation*

Let's consider training via MLE: say we have a batch of *data samples*

$$\mathbb{D} = \{x^j \sim P_{\text{data}} : j = 1, \dots, n\}$$

In MLE we maximize *log-likelihood* which is

$$\begin{aligned}\max_{\theta} \log \mathcal{L}(\theta) &= \max_{\theta} \frac{1}{n} \sum_j \log P_{\theta}(x^j) \\ &= \max_{\theta} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \{\log P_{\theta}(x)\} \\ &= \max_{\theta} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \left\{ \log \int P_{\theta}(x|z) P(z) \mathrm{d}z \right\}\end{aligned}$$

to handle this challenging computation, we developed *ELBO*

MLE Training via *Variational Inference*

Computational ELBO

Variational inference indicates that

$$\log P_{\theta}(x^j) = \max_{\mathbf{w}} \text{ELBO}_{\theta, x^j}(Q_{\mathbf{w}})$$

for some *computational model* $Q_{\mathbf{w}}(z|x^j)$

- + Why do we index ELBO by θ and x^j ?
- Well! Just write the ELBO down

Recall that ELBO is given by

$$\text{ELBO}_{\theta, x^j}(Q_{\mathbf{w}}) = \mathbb{E}_{z \sim Q_{\mathbf{w}}} \{ \log P_{\theta}(x^j|z) \} - D_{\text{KL}}(Q_{\mathbf{w}} \| \mathcal{N}^0)$$

which obviously depends on the generator P_{θ} and data sample x^j

MLE Training via *Variational Inference*

Let us now represent sample *ELBO estimate computed at x* as

$$\text{ELBO}(\mathbf{w}, \theta | x) = \hat{\mathbb{E}}_{z \sim Q_{\mathbf{w}}} \{\log P_{\theta}(x|z)\} - D_{\text{KL}}(Q_{\mathbf{w}} \| \mathcal{N}^0)$$

We maximize this sample *ELBO* over $\mathbf{w}$ to find log-likelihood; thus, MLE is

$$\begin{aligned} \max_{\theta} \log \mathcal{L}(\theta) &= \max_{\theta} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \{\log P_{\theta}(x)\} \\ &= \max_{\theta} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \left\{ \max_{\mathbf{w}} \text{ELBO}(\mathbf{w}, \theta | x) \right\} \end{aligned}$$

Let's think algorithmic about computing log-likelihood

Full Loop via *Variational Inference*

$$\log \mathcal{L}(\theta) = \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \left\{ \max_{\mathbf{w}} \text{ELBO}(\mathbf{w}, \theta | x) \right\}$$

Compute_LL($\mathbf{w}, \theta$):

```

1: Sample a batch of data samples  $\{x^j : j = 1, \dots, n\}$ 
2: for  $j = 1, \dots, n$  do
3:   Sample a batch of latent samples  $\{z^\ell : \ell = 1, \dots, r\}$  from  $Q_{\mathbf{w}}$ 
4:   Compute ELBO( $\mathbf{w}, \theta | x^j$ ) over latent batch
5:   for  $T$  iterations do
6:     Iterate with  $\nabla_{\mathbf{w}} \text{ELBO}(\mathbf{w}, \theta | x^j)$  to maximize ELBO at  $\mathbf{w}^*$ 
7:   end for
8:   Set  $\text{ELBO}^j \leftarrow \text{ELBO}(\mathbf{w}^*, \theta | x^j)$ 
9: end for
10: Estimate log-likelihood as  $\log \mathcal{L} \leftarrow \text{mean} \{\text{ELBO}^j\}$ 
11: return log-likelihood estimate  $\log \mathcal{L}$ 

```

This means that *log-likelihood* optimization requires a *two-tier* loop

↳ This is computationally *expensive*

Approximate Loop via *Variational Inference*

Using Jensen's inequality: we can write

$$\hat{\mathbb{E}}_{x \sim P_{\text{data}}} \left\{ \max_{\mathbf{w}} \text{ELBO}(\mathbf{w}, \theta | x) \right\} \geq \max_{\mathbf{w}} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \{ \text{ELBO}(\mathbf{w}, \theta | x) \}$$

This means that the MLE loop can be bounded as

$$\begin{aligned} \max_{\theta} \log \mathcal{L}(\theta) &= \max_{\theta} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \left\{ \max_{\mathbf{w}} \text{ELBO}(\mathbf{w}, \theta | x) \right\} \\ &\geq \max_{\theta} \max_{\mathbf{w}} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \{ \text{ELBO}(\mathbf{w}, \theta | x) \} \end{aligned}$$

Hoping that this bound is *tight enough*, we could simplify the MLE loop as

$$\max_{\theta} \log \mathcal{L}(\theta) \approx \max_{\theta} \max_{\mathbf{w}} \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \{ \text{ELBO}(\mathbf{w}, \theta | x) \}$$

Approximate Loop via *Variational Inference*

Approx_Compute_LL($\mathbf{w}, \theta$):

```

1: Sample a batch of data samples  $\{x^j : j = 1, \dots, n\}$ 
2: for  $j = 1, \dots, n$  do
3:   Sample a batch of latent samples  $\{z^\ell : \ell = 1, \dots, r\}$  from  $Q_{\mathbf{w}}$ 
4:   Compute ELBO ( $\mathbf{w}, \theta | x^j$ ) over latent batch
5:   Set  $\text{ELBO}^j \leftarrow \text{ELBO}(\mathbf{w}, \theta | x^j)$            #no inner optimization loop
6: end for
7: Estimate log-likelihood as  $\log \mathcal{L} \leftarrow \text{mean} \{ \text{ELBO}^j \}$ 
8: return log-likelihood estimate  $\log \mathcal{L}$ 

```

This makes *log-likelihood* optimization a classical risk minimization with

$$\hat{R}(\mathbf{w}, \theta) = \hat{\mathbb{E}}_{x \sim P_{\text{data}}} \{ -\text{ELBO}(\mathbf{w}, \theta | x) \}$$

which can be readily performed by a single gradient iteration loop

Completing Computational Model

Though the training is formulated: *there are a few questions*

- + How should we build the model $Q_{\mathbf{w}}(z|x)$?
- It needs to be a **probability distribution** computed from a sample x
- It should **marginalize** to **prior latent distribution**
- + How **easy** is it to compute the required **sample gradients**, i.e.,

$$\nabla_{\mathbf{w}} \text{ELBO}(\mathbf{w}, \theta | x) \quad \text{and} \quad \nabla_{\theta} \text{ELBO}(\mathbf{w}, \theta | x)$$

- Well! They are mostly **trivial** except some **tricky parts**